

AI Harness 作業預覽

本檔整合以下兩個文件部分，方便先做內容檢查：

- 第一部分：書面報告
- 第三部分：`log.md`

第一部分：書面報告

題目

程式碼維運代理：軟體維護用 AI Harness 系統設計

摘要

本報告提出一個用於軟體維護的 AI Harness 系統，名稱為「程式碼維運代理」(Code Operations Agent)。此系統面向 CI 失敗排查、回歸錯誤診斷、缺陷分析與修補規劃等維運工作。設計重點不在模型訓練，而在於如何讓大型語言模型擔任系統控制器，透過 function calling 呼叫外部工具、結合記憶機制，並遵循多步驟 workflow 產生具證據基礎的建議。本文提出三個核心工具、一個混合式 orchestration 機制，以及一套離線 evaluation 方法。系統目標是縮短 incident triage 時間、提升診斷一致性，並讓維運決策流程更可解釋。

1. 問題定義與背景

現代軟體團隊在處理 CI 失敗、版本回歸或執行期錯誤時，往往需要同時查閱 issue tracker、CI logs、pull request 與 source code。真正耗時的部分通常不是最後修改程式碼，而是先判斷要看哪裡、哪個訊號最可信、下一步該查什麼。這類工作高度依賴工程師經驗，因此相似 incident 可能被不同人以截然不同的速度與品質處理。

本設計選擇的應用場景是軟體維護用的程式碼維運代理。目標使用者包含 on-call engineers、一般軟體工程師，以及需要快速掌握狀況的 tech leads。此系統的核心任務是協助使用者蒐集上下文、縮小問題範圍、形成 root-cause hypothesis，並提出安全的修補方向。

系統範圍刻意限制在 triage、診斷與 patch planning，不直接執行 merge、deploy 或其他破壞性 repository 操作。這樣的邊界有助於安全性與可解釋性。系統設計目標是在 90 秒內提供第一個可執行的診斷建議，並控制工具呼叫成本，同時避免在 prompts 或摘要中暴露敏感資訊。

2. AI Harness 系統架構

程式碼維運代理由四個主要層次構成：LLM controller、tool layer、memory layer 與 orchestration layer。

LLM controller 是整個系統的核心控制元件，負責解析使用者請求、判斷是否需要呼叫工具、整合中間結果，並輸出最後回應。也就是說，LLM 在這裡不是一般聊天模型，而是受控的 system controller。

Tool layer 提供 LLM 無法單獨可靠完成的外部能力。本設計中包含三個核心工具。第一個是 Issue/PR Context Fetcher，用來取得 issue metadata、相關 pull requests、ownership 與近期討論內容。第二個是 CI Log Analyzer，用來從 workflow logs 中萃取 failure signature 與可能原因。第三個是 Repo Code Search plus Diff Planner，用來定位可疑程式碼區域並提出最小化修補方案。

Memory layer 分為短期與長期記憶。短期記憶保存當前案件狀態，例如 incident identifier、目前假設、已執行的工具呼叫與 confidence。長期記憶則儲存重複出現的 incident signature、已驗證修復方法與特定服務的維運知識。

Orchestration layer 負責整體流程控制，包括 state transition、stop conditions、guardrails 與 escalation 規則。整體資料流如下：使用者提出請求後，controller 先進行 intent classification，接著建立 tool plan，然後呼叫工具、整合證據，最後產生建議。這種顯式結構比完全自由式的 agent loop 更容易除錯與評估。

3. Function Calling 與 Tool Usage 設計

Function calling 是本系統把使用者意圖轉換成實際行動的核心機制。每個工具都必須有清楚的目的、input schema、output schema、觸發條件與 fallback 策略。

Issue/PR Context Fetcher 會在使用者提到 bug ticket、incident ID 或 pull request 時啟動。輸入包含 repository 名稱、issue 或 PR identifier，以及時間範圍。輸出則包含標題、狀態、參與者與近期事件。此工具可在深入診斷前提供必要的 operational context。

CI Log Analyzer 會在任務涉及 failed build、failed test 或 deployment error 時觸發。其輸入為 repository、workflow run ID 與 job ID。輸出內容包括 failure signature、關鍵 error lines、可能原因與 confidence score。這個工具的價值在於把 raw logs 壓縮成結構化訊號，降低 LLM 直接閱讀長日誌的負擔。

Repo Code Search plus Diff Planner 會在 controller 已形成初步假設後啟動，用來尋找 source-level evidence。輸入為 repository、branch 與 search query；輸出則為 candidate files、suspect functions 與 patch plan。這個工具不是要取代工程師寫 patch，而是要把排查範圍縮小到最可能的程式碼區域。

所有工具輸出都必須先通過 schema validation。若工具 timeout，controller 會以更小範圍重試一次；若輸出格式錯誤，controller 會要求正規化輸出並重試一次。系統預設為 read-only，所有涉及修改程式碼的內容都屬於 advisory output，必須先經過人工確認。

4. 多步驟 Agent Workflow

本系統的 workflow 可分為五個主要階段。第一階段是 intent understanding。Controller 先將使用者請求分類為 CI failure triage、regression diagnosis、runtime bug investigation 或 patch planning 等類型。第二階段是 planning，controller 會建立有順序的 tool-call plan、設定工具預算，並定義 stop conditions。第三階段是 tool execution。以典型 CI incident 為例，controller 會先呼叫 Issue/PR Context Fetcher 取得背景，再呼叫 CI Log Analyzer 取得 failure signature，最後使用 Repo Code Search plus Diff Planner 進程式碼定位與修補建議。

第四階段是 evidence integration。Controller 會比較 issue context、logs 與 repository evidence 之間是否一致。如果三方訊號互相支持，就能形成高信心診斷；若訊號彼此矛盾，系統可以追加一次工具呼叫、降低 confidence，或直接進入 escalation。第五階段是 final response generation，輸出內容包含可能 root cause、confidence score、最相關的 files 或 functions、修補方向與風險事項。

Workflow 也包含 branch 與 recovery logic。如果 logs 顯示 flaky test 或非決定性失敗，controller 會走 stability branch，而不是直接進入 patch proposal branch。若錯誤屬於 deterministic compilation failure，

則可以直接朝修補規劃前進。若工具 timeout，系統會縮小範圍重試；若 schema 不合法，系統會先修正格式再重試。案件完成後，系統會把 incident signature 與接受的解法寫入 memory。

5. Orchestration 與 Decision Logic

本設計採用混合式 orchestration。State-machine 負責規定可接受的流程轉移，而 LLM 則在每個 state 內做上下文推理。這種做法兼顧了控制能力與彈性。

系統有三個核心 decision checkpoints。第一，檢查使用者請求是否足夠明確，能否安全展開工具呼叫。第二，檢查目前蒐集到的證據是否足以支持診斷。第三，檢查 confidence 是否高到可以提出 patch recommendation，而不只是保留在假設層級。

此外，系統也有清楚的 stop conditions。當證據覆蓋率足夠、工具預算耗盡，或使用者要求轉交人工時，流程就應終止。若 confidence 過低或證據彼此衝突，controller 會升級到 human reviewer，並輸出 compact evidence pack，包括取得的 context、主要 failure signal、衝突線索與建議下一步。

6. 評估方法

本系統的評估方式採用離線 historical case set，包含 30 筆軟體維護 incidents，並平衡涵蓋 CI failures、runtime bugs 與 regressions。每一筆案例都應附帶足夠的 repository context、issue 資訊與 logs，讓整個決策流程可以重現。

主要評估指標共有五項：task completion rate、tool-call accuracy、root-cause precision@1、mean time to first actionable recommendation，以及 failure recovery success rate。評分可以使用 0 到 2 分的尺度，從 diagnosis quality、actionability 與 safety compliance 三個面向給分。Baseline 則可以設計成不具 tool access 的 LLM-only assistant，用來比較 orchestration 與 tool use 帶來的增益。

系統優化流程採標準工程方式：定期回顧低分案例、檢查 routing 或 evidence integration 哪裡失效、調整 tool prompts 與 routing rules，然後重新執行 evaluation set。

7. 討論與結論

程式碼維運代理的最大優勢在於，它把軟體維護視為 workflow 問題，而不是純粹的文字生成問題。透過明確的工具與 orchestration，系統可以減少無根據猜測，並為工程師提供一條可追蹤、可審查的診斷路徑。

本設計也有其限制。系統品質仍依賴工具穩定性、repository context 的新鮮度，以及長期記憶的覆蓋率。另外，如果 schema 設計不佳或 escalation threshold 設太寬，依然可能產生過度自信的建議。這些風險可透過 read-only 預設、confidence gating、schema validation 與 human approval 來部分緩解。

總結而言，本報告提出一個面向軟體維護工作的實用 AI Harness 設計。程式碼維運代理讓 LLM 擔任 controller，結合結構化工具、記憶層與混合式 orchestration，形成一個邏輯一致、可解釋且符合實務需求的系統。此設計符合本作業目標，因為它清楚展示了 AI 如何在不以模型訓練為主軸的前提下，透過 tool use、workflow control 與 decision-making 來完成複雜任務。

第三部分：log.md

專案資訊

- 開始日期：2026-05-21
- 作業類型：AI Harness system design
- 主題：Code Operations Agent (程式碼維運代理)

紀錄格式

每筆紀錄使用以下欄位：

- 時間：
- 提示 / 需求：
- AI 回應摘要：
- 設計決策：
- 發現問題：
- 修正 / 調整：
- 下一步：

Entry 001

- 時間：2026-05-21
- 提示 / 需求：將原本的 DRL 作業工作區改成新的 AI Harness 作業規格。
- AI 回應摘要：將原本以 DRL 為主的 instruction 與 TODO 結構改寫為 AI Harness 版本。
- 設計決策：將專案主軸從演算法 survey 轉為 system orchestration design。
- 發現問題：原有檔案內容完全對應舊題目，已不符合新的評分方向。
- 修正 / 調整：重寫 `instruction.md` 與 `todo.md`，並建立 `log.md`。
- 下一步：選定一個應用場景並開始架構草稿。

Entry 002

- 時間：2026-05-27
- 提示 / 需求：確認主題選擇為「Code Operations Agent」。
- AI 回應摘要：將規劃檔案改為軟體維護型 AI Harness 設計。
- 設計決策：
- 選擇以 CI triage、錯誤診斷與 patch planning 為核心的程式碼維運場景。
- 採用 hybrid orchestration (state-machine + LLM controller)。
- 定義三個核心工具與 schema-driven function calling。
- 發現問題：
- 既有模板仍過於通用，尚未綁定特定應用場景。
- 修正 / 調整：
- 更新 `todo.md` 的 Step 0-6 進度。
- 填寫 `report/outline.md` 的場景化內容。
- 重寫 `figures/infographic_plan.md` 的版面與流程。
- 新增 `notes/code_ops_agent_design.md` 作為架構與評估筆記。

- 下一步：
- 依大綱撰寫 2-5 頁正式報告。
- 依資訊圖規劃製作圖稿。

Entry 003

- 時間：2026-06-03
- 提示 / 需求：撰寫書面報告初稿。
- AI 回應摘要：將大綱擴寫成 Code Operations Agent 的完整報告初稿。
- 設計決策：
- 報告重點放在 system design、tool orchestration 與 workflow control。
- 章節結構直接對齊作業評分項目。
- 為了安全性，將 deploy 與 merge 類動作排除在系統範圍外。
- 發現問題：
- 專案雖然已有設計筆記與大綱，但尚未有可提交的報告正文。
- 修正 / 調整：
- 新增 `report/report.md`。
- 同步更新 `todo.md`，標記報告初稿相關項目已完成。
- 下一步：
- 做最後的文字潤稿與頁數控制。
- 將資訊圖規劃轉成最終圖稿內容。

Entry 004

- 時間：2026-06-03
- 提示 / 需求：潤飾英文初稿、製作中文版本，並完成資訊圖最終文字稿。
- AI 回應摘要：新增較正式的英文版報告、中文版報告，以及資訊圖最終文案檔。
- 設計決策：
- 保留原始英文初稿，另做一份更正式的英文版。
- 中文版不採逐句直譯，而是以可直接交件為目標重寫。
- 將資訊圖規劃改寫為 panel-by-panel 的最終內容，方便排版。
- 發現問題：
- 專案已有不錯的初稿與規劃，但仍缺少更接近最終展示用途的文字包。
- 修正 / 調整：
- 新增 `report/report_polished.md`。
- 新增 `report/report_zh.md`。
- 新增 `figures/infographic_final.md`。
- 更新 `todo.md`，反映資訊圖內容已具備。
- 下一步：
- 針對選定語言版本做頁數壓縮。
- 將資訊圖文案真正排成圖。

Entry 005

- 時間：2026-06-06
- 提示 / 需求：完成最後三項收尾工作，包括精簡中文版、單頁資訊圖文案與 submission checklist。
- AI 回應摘要：產出較短的中文終稿、一頁式中文資訊圖文案，以及獨立的 final submission checklist。
- 設計決策：
- 額外保留一份精簡中文版，作為最適合提交的版本。
- 將資訊圖內容整理成 presentation-ready blocks，而不只是規劃筆記。
- 增加一份獨立 checklist，讓最後交件整理更明確。
- 發現問題：
- 專案內容雖完整，但最後交件相關檔案仍分散在不同資料夾。
- 修正 / 調整：
- 新增 `report/report_zh_final.md`。
- 新增 `figures/infographic_onepage_zh.md`。
- 新增 `notes/submission_checklist.md`。
- 更新 `todo.md`，標記校稿、資訊圖文案與 log 品質檢查已完成。
- 下一步：
- 若課程有嚴格命名規則，需再調整檔名。
- 若老師要求非 Markdown 格式，需再匯出成最終格式。

Entry 006

- 時間：2026-06-06
- 提示 / 需求：將最終交件內容整理到 `summiton` 資料夾。
- AI 回應摘要：把最終報告、資訊圖文案與 log 集中成一個交件包。
- 設計決策：
- 以精簡中文版報告作為主要報告交件版本。
- 以單頁中文資訊圖文案作為資訊圖交件文字稿。
- 保留使用者指定的資料夾拼法：`summiton`。
- 發現問題：
- 最終交件檔案原本散落在不同目錄，交付時不夠直觀。
- 修正 / 調整：
- 建立 `summiton/`。
- 將三個主要交件檔複製進去，並加上排序檔名。
- 新增 `summiton/README.md` 說明交件包內容。
- 下一步：
- 若課程有指定檔名規則，再進一步重新命名。
- 若需要 PDF 或圖片格式，再進行匯出或轉檔。

Entry 007

- 時間：2026-06-06
- 提示 / 需求：重新檢查資訊圖內容是否符合原始作業要求，修正內容後直接產出圖檔。
- AI 回應摘要：重寫資訊圖文案結構，刪除過度分散注意力的內容，並產出一份可直接提交的 SVG 資訊圖。
- 設計決策：
- 讓資訊圖聚焦在四個作業要求：architecture、workflow、function calling、sequence/pipeline。
- 把 problem statement 與過重的敘述性內容降到最低，避免搶走核心系統設計版面。
- 以 SVG 形式產出，方便本地直接提交或後續匯出。
- 發現問題：
- 舊版資訊圖文案雖然方向正確，但版面焦點分散，對作業要求的對應不夠直接。
- 交件資料夾中原本只有資訊圖文字稿，沒有真正的圖檔。
- 修正 / 調整：
- 重寫 `figures/infographic_final.md`。
- 重寫 `figures/infographic_onepage_zh.md`。
- 新增 `figures/code_ops_agent_infographic.svg`。
- 複製圖檔到 `summiton/02_infographic.svg`。
- 更新 `summiton/README.md`。
- 下一步：
- 若老師要求 PNG 或 PDF，再從 SVG 匯出對應格式。